

Informe PC4 - Desarrollo de Software

Curso: CC3S2 Desarrollo de Software **Alumna:** Melissa Iman Noriega **Fecha:** 01/07/2026 **Repositorio:** <https://github.com/Melissa1221/pc4-desarrollo-software>

1. Descripción

Se desarrolló el núcleo de dominio en Java (Maven) de una plataforma de rescate y adopción de mascotas. El trabajo cubre los tres módulos del enunciado y se centra en los patrones de diseño usados en cada uno, tal como se pidió priorizar. No se implementó interfaz gráfica ni base de datos; los datos se manejan en memoria para poder demostrar los patrones.

2. Requisitos cumplidos

Módulo 1 — Reporte de animales perdidos y alertas

- RF 1.1: se registra una mascota perdida con nombre, especie, raza, foto y descripción (LostPet).
- RF 1.2: el reporte guarda las coordenadas geográficas (lat, lon).
- RF 1.3: un ciudadano anónimo registra un avistamiento con foto y ubicación (Sighting).
- RF 1.4: al reportar una mascota se notifica a los suscriptores dentro de un radio configurable (AlertCenter.reportLostPet).
- RNF 1.1 (latencia): la notificación es una llamada directa en memoria, sin overhead de red.
- RNF 1.2 (anonimato): el id del dueño no se expone en la alerta que reciben los ciudadanos.

Módulo 2 — Buscador por imagen

- RF 2.1: se carga una imagen para la búsqueda (ImageQuery).
- RF 2.2: se obliga a elegir una de tres intenciones (SearchIntent).
- RF 2.3: adopción devuelve solo protectoras/ONGs (AdoptionSearch).
- RF 2.4: venta devuelve solo criaderos certificados (SaleSearch).
- RF 2.5: verificar pérdida contrasta con las alertas activas (LostCheckSearch).
- RNF 2.1 (abstracción): los metadatos van en un mapa clave-valor (JSON plano), de modo que el motor de búsqueda es intercambiable.

Módulo 3 — Red de cuidadores

- RF 3.1: tres roles de cuidador (solidario, profesional, especializado).
- RF 3.2: cada cuidador define sus restricciones (especies, tamaños, medicación).
- RF 3.3: interruptor para activar/desactivar las alertas del módulo 1 (toggleAlerts).
- RF 3.4: el perfil guarda el estado de verificación para habilitarlo públicamente.
- RNF 3.1 (confiabilidad): el perfil solo se hace público si el documento fue validado (isPublic).

3. Patrones de diseño usados

Patrón	Dónde	Para qué
Singleton	AlertCenter	un único registro central de alertas
Observer	AlertObserver, AlertCenter	notificar a los cuidadores suscritos en un radio (RF 1.4)
Strategy	SearchStrategy y sus tres implementaciones	cada intención resuelve la búsqueda a su manera (RF 2.2–2.5)

Patrón	Dónde	Para qué
Factory	SearchStrategyFactory	crear la estrategia correcta según la intención
Builder	CaregiverProfileBuilder	armar el perfil con restricciones opcionales (RF 3.2)
Decorator	RoleDecorator y los tres roles	sumar capacidades por rol (RF 3.1)

Los patrones se tomaron como referencia del repositorio visto en clase (peterm85/design-patterns), respetando su estructura de interfaz + clases concretas.

4. Arquitectura

El código está separado por módulo en tres paquetes, cada uno independiente:

```
com.uni.petfinder
├── alertas      → módulo 1 (Observer + Singleton)
├── busqueda    → módulo 2 (Strategy + Factory)
├── cuidadores  → módulo 3 (Builder + Decorator)
└── App         → demostración de los tres módulos
```

La separación en paquetes responde al RNF 3.2 (escalabilidad): el servicio de cuidadores no depende del de alertas, así que un pico de demanda en uno no afecta al otro. La comunicación entre módulos es por interfaces, lo que permite reemplazar implementaciones sin tocar el resto.

5. Cómo ejecutar

```
mvn compile
mvn exec:java
```

La clase App ejecuta los tres módulos e imprime el resultado de cada patrón.